

Analyse du système de recommandation musicale

Vue d'ensemble du système

Cette application Flask crée un système de recommandation musicale personnalisé basé sur un questionnaire contextuel. Elle utilise un dataset enrichi de chansons avec de nombreuses métadonnées pour suggérer des morceaux adaptés à l'humeur et au contexte de l'utilisateur.

Structure des données

Le dataset contient **40 colonnes** avec différents types d'informations :

Informations de base

- `id` : Identifiant unique (probablement Spotify ID)
- `artiste`, `titre`, `genre` : Métadonnées basiques
- `macro_genre` : Classification générale du genre

Contextes utilisateur (6 dimensions)

- `contexte_emotionnel` : État émotionnel (joyeux, calme, etc.)
- `contexte_meteo` : Conditions météo (ensoleillé, pluvieux, etc.)
- `contexte_activite` : Activité en cours (sport, détente, travail, etc.)
- `contexte_lieu` : Lieu (maison, voiture, café, etc.)
- `contexte_social` : Contexte social (seul, amis, famille, etc.)
- `contexte_temporel` : Moment (matin, soir, nuit, etc.)

Features audio Spotify

- `bpm_estime` : Battements par minute
- `energie` : Niveau d'énergie (0-1)
- `valence` : Positivité/négativité (0-1)
- `danceability` : Aptitude à danser (0-1)
- `acousticness`, `instrumentalness`, `speechiness`, `liveness` : Autres caractéristiques audio

Scores calculés et catégories

- `mood_score`, `mood_category` : Score d'humeur
- `party_factor`, `party_category` : Facteur fête
- `organic_score`, `organic_category` : Score d'authenticité
- `sport_suitability`, `relax_suitability`, `study_suitability` : Aptitudes spécialisées
- `diversity_index`, `diversity_category` : Index de diversité
- `context_popularity_score`, `popularity_category` : Popularité contextuelle

Fonctionnement étape par étape

1. Initialisation de l'application

```
# Chargement du dataset
df = pd.read_csv("tracks_dataset_enriched.csv")
df.set_index('id', inplace=True) # ID comme index (Spotify ID)

# Initialisation du gestionnaire Spotify
spotify_manager = SpotifyManager()

# Configuration des thèmes visuels
THEMES = {
    "joyeux": {"primary": "#9933CC", "secondary": "#CC66FF", ...},
    "energique": {"primary": "#FF00FF", "secondary": "#FF66FF", ...},
    # ... autres thèmes
}
```

2. Questionnaire contextuel

6 questions couvrant les dimensions contextuelles :

```
QUESTIONS = [
    ("🗨️ Comment vous sentez-vous ?", "emotionnel", ["joyeux", "energique",
"calme", ...]),
    ("☀️ ☁️ Quel temps fait-il ?", "meteo", ["ensoleille", "nuageux",
"pluvieux", ...]),
    ("🏃🏠 Que faites-vous en ce moment ?", "activite", ["sport", "detente",
"travail", ...]),
    ("📍 Où êtes-vous ?", "lieu", ["maison", "plage", "voiture", ...]),
    ("👥 Avec qui êtes-vous ?", "social", ["seul", "amis", "famille",
...]),
    ("🕒 Quel moment ?", "temporel", ["matin", "apres_midi", "soiree",
...])
]
```

3. Système de scoring sophistiqué

Fonction `calculate_score(song, answers)`

Score total sur 100 points répartis en 4 catégories :

A. Score contextuel (50 points)

```
context_weights = {
    'emotionnel': 20, # Le plus important
    'activite': 15,   # Très important
    'meteo': 5,      # Influence modérée
    'lieu': 4,        # Influence faible
    'social': 3,      # Influence faible
    'temporel': 3     # Influence faible
}
```

Pour chaque contexte, si la réponse utilisateur correspond exactement au contexte de la chanson, le score est ajouté.

B. Score audio features (25 points)

Basé sur l'humeur déclarée, analyse des caractéristiques audio :

- **Valence** (8 points) : Positivité selon l'humeur
 - Joyeux/Énergique : valence élevée (0.6-1.0)
 - Nostalgique : valence faible (0.0-0.4)
 - Calme/Romantique : valence moyenne
- **Énergie** (8 points) : Niveau d'énergie selon l'humeur et le temps
 - Énergique : énergie élevée (0.7-1.0)
 - Calme : énergie faible (0.0-0.5)
- **Danceability** (6 points) : Aptitude à danser adaptée selon l'activité et le contexte social
- **BPM** (3 points) : Tempo selon l'activité
 - Sport : 120-180 BPM
 - Détente : 60-100 BPM
 - Fête : 110-140 BPM

C. Scores spécialisés (20 points)

Utilise les scores précalculés du dataset :

- `sport_suitability` pour l'activité "sport"
- `relax_suitability` pour "détente"
- `study_suitability` pour "étude/travail"
- `party_factor` pour "fête"
- `mood_score` général

D. Bonus (5 points)

- `context_popularity_score` : Popularité contextuelle
- `diversity_index` : Diversité musicale
- `organic_score` : Authenticité

4. Algorithme de recommandation

Fonction `get_recommendations(df, answers, exclude_song_indices, num=6)`

Processus de sélection :

1. **Filtrage initial**
 - Exclusion des chansons déjà recommandées
 - Filtrage du contenu offensant
2. **Calcul des scores**
 - Score de correspondance pour chaque chanson
 - Détection des "perfect matches" (correspondance exacte sur tous les contextes)
3. **Tri intelligent**
4. `sorted_candidates = sorted(`
5. `all_candidate_songs,`
6. `key=lambda x: (x['is_perfect_match'], x['matches'],`
7. `x.get('bpm_estime', 0), x['score']),`
7. `reverse=True`
8. `)`
 - Perfect matches en premier
 - Puis par nombre de correspondances contextuelles

- Puis par BPM (énergie)
- Enfin par score global

9. Diversité artistique

- Évite trop de chansons du même artiste
- Équilibre entre qualité et diversité

5. Fonction de correspondance parfaite

Fonction check_perfect_match(song, answers)

Vérifie si une chanson correspond **exactement** à toutes les réponses de l'utilisateur :

```
def check_perfect_match(song, answers):
    matches = 0
    total_contexts = 0

    for context_type, user_answer in answers.items():
        col_name = f"contexte_{context_type}"
        if col_name in song.index:
            total_contexts += 1
            if str(song[col_name]).lower() == str(user_answer).lower():
                matches += 1

    return matches == total_contexts and total_contexts > 0, matches,
    total_contexts
```

6. Gestion des playlists

Fonctionnalités :

- Création de playlists personnalisées
- Ajout/suppression de chansons
- Évitement des doublons
- Export en fichier texte

Vérification anti-doublons :

```
def is_song_in_playlist(song_index, playlist_songs):
    for existing_song in playlist_songs:
        if existing_song.get('original_index') == song_index:
            return True
    return False
```

7. Intégration Spotify

Via la classe `SpotifyManager` :

- Recherche d'informations détaillées
- Liens vers Spotify
- Métadonnées enrichies

Avantages du système

1. **Approche multicritère** : Combine contexte, audio features et scores spécialisés
2. **Pondération intelligente** : L'émotion et l'activité ont plus de poids
3. **Perfect matches** : Priorise les correspondances exactes
4. **Diversité** : Évite la monotonie artistique
5. **Persistance** : Évite les recommandations répétitives dans une session
6. **Personnalisation** : Thèmes visuels selon l'humeur

Architecture technique

- **Backend** : Flask avec sessions pour la persistance
- **Frontend** : Templates Jinja2 avec thèmes dynamiques
- **Données** : Pandas pour le traitement du dataset
- **API** : Intégration Spotify pour métadonnées
- **Export** : Génération de fichiers texte pour les playlists

Ce système combine l'analyse contextuelle humaine avec les caractéristiques audio techniques pour créer des recommandations musicales très personnalisées.

Pourquoi cet ordre est meilleur :

L'activité avant le temporel car :

- **L'activité détermine le type de musique** : sport = énergique, détente = calme, travail = concentration
- **Le BPM dépend de l'activité** : sport (120-180), détente (60-100), travail (80-120)
- **Les scores spécialisés** (sport_suitability, relax_suitability) sont directement liés à l'activité

Le temporel en dernier car :

- Il est plus **contextuel que déterminant** pour le choix musical
- Une chanson énergique peut convenir le matin ET le soir selon l'activité
- L'émotion et l'activité sont plus **prédictives** du goût musical

Exemple concret : Pour "sport + soir", l'algorithme privilégiera d'abord une musique énergique (activité) plutôt qu'une musique calme (temporel), ce qui est plus pertinent.

L'ordre du code actuel est donc **musicalement plus logique**.

D'après le code que vous avez partagé, voici une explication complète du système :

But du système

Il s'agit d'un **système de recommandation musicale intelligent** qui propose des chansons personnalisées en fonction du contexte et de l'état émotionnel de l'utilisateur. Le système analyse la situation actuelle de l'utilisateur (humeur, activité, environnement) pour suggérer la musique la plus adaptée.

Comment ça fonctionne

1. Questionnaire contextuel

L'utilisateur répond à 6 questions courtes :

-  **Émotionnel** : Comment vous sentez-vous ? (joyeux, énergique, calme, nostalgique, romantique, neutre)
-   **Météo** : Quel temps fait-il ? (ensoleillé, nuageux, pluvieux, etc.)
-  **Activité** : Que faites-vous ? (sport, détente, travail, fête, cuisine, etc.)
-  **Lieu** : Où êtes-vous ? (maison, plage, voiture, café, etc.)
-  **Social** : Avec qui ? (seul, amis, famille, groupe, couple)
-  **Moment** : Quelle heure ? (matin, après-midi, soirée, nuit)

2. Algorithme de recommandation sophistiqué

Le système utilise un scoring complexe (sur 100 points) qui combine :

- **Correspondance contextuelle** (50 points) : Match entre les réponses et les contextes des chansons
- **Caractéristiques audio** (25 points) : Analyse de la valence, énergie, danceability, BPM
- **Scores spécialisés** (20 points) : Aptitude pour le sport, la détente, l'étude, la fête
- **Bonus** (5 points) : Popularité, diversité, authenticité

3. Fonctionnalités avancées

- **Intégration Spotify** : Liens directs vers les morceaux
- **Système de playlists** : Création et gestion de listes personnalisées
- **Export** : Sauvegarde des playlists au format texte
- **Interface adaptative** : Thème visuel qui change selon l'humeur
- **Filtrage** : Exclusion du contenu offensant

Public cible

Utilisateurs principaux :

- **Mélobanes** recherchant une découverte musicale personnalisée
- **Personnes actives** voulant la musique parfaite pour chaque moment (sport, travail, détente)

- **Utilisateurs de streaming** (Spotify) cherchant à enrichir leurs playlists
- **Jeunes adultes et adultes** sensibles au contexte et à l'ambiance

Cas d'usage typiques :

- 🏃 **Sportifs** : Musique motivante pour l'entraînement
- 📖 **Étudiants/Travailleurs** : Musique de concentration
- 🎉 **Fêtards** : Ambiance pour les soirées
- 🧘 **Détente** : Musique relaxante après le travail
- 🚗 **Conducteurs** : Playlist adaptée aux trajets
- 👫 **Couples** : Musique romantique pour les moments intimes

Avantages compétitifs

1. **Contextualisation poussée** : Va au-delà des goûts musicaux traditionnels
2. **Simplicité d'usage** : 6 questions rapides vs algorithmes complexes
3. **Pertinence situationnelle** : Adapte la musique au moment présent
4. **Diversité** : Évite la répétition et favorise la découverte
5. **Intégration native** : Connexion directe avec Spotify

Ce système cible donc les utilisateurs qui veulent une expérience musicale **intelligente, contextuelle et personnalisée**, particulièrement adaptée à notre époque où la musique accompagne chaque moment de la vie quotidienne.

Principe général

L'application pose 6 questions contextuelles à l'utilisateur, puis utilise un algorithme de scoring avancé pour recommander les chansons les plus adaptées à son état d'esprit et sa situation actuelle.

Les fonctions clés expliquées

1. `calculate_score(song, answers)` - Le cœur du système

Cette fonction attribue un score sur 100 points à chaque chanson en analysant 4 dimensions :

- **Contexte** (50 pts) : Compare les réponses utilisateur aux colonnes `contexte_*` du dataset
- **Audio** (25 pts) : Analyse `valence`, `energie`, `danceability`, `bpm_estime` selon l'humeur
- **Spécialisé** (20 pts) : Utilise les scores précalculés comme `sport_suitability`, `party_factor`
- **Bonus** (5 pts) : Récompense la popularité contextuelle et la diversité

2. `check_perfect_match(song, answers)` - Détection des correspondances parfaites

Identifie les chansons qui correspondent **exactement** à toutes les réponses utilisateur. Ces chansons sont priorisées dans les recommandations.

3. `get_recommendations(df, answers, exclude_song_indices, num=6)` - Algorithme de sélection

- Filtre les chansons offensantes et déjà recommandées
- Calcule les scores pour toutes les chansons
- Trie par : perfect matches → nombre de correspondances → BPM → score global
- Applique une diversité artistique (évite trop de chansons du même artiste)

Workflow utilisateur

1. **Questionnaire** : 6 questions sur l'émotion, météo, activité, lieu, contexte social, moment
2. **Scoring** : Chaque chanson du dataset reçoit un score personnalisé
3. **Recommandations** : 6 chansons optimales sont sélectionnées
4. **Playlists** : L'utilisateur peut sauvegarder ses favoris
5. **Nouvelles suggestions** : Génération de nouvelles recommandations (sans répétition)

Le système est particulièrement intelligent car il combine les données contextuelles humaines avec les caractéristiques audio techniques de Spotify pour créer des recommandations très précises et personnalisées.

Exactement ! Vous avez tout à fait raison. Dans ce contexte, **BPM signifie "Beats Per Minute"** (battements par minute) et c'est une mesure musicale qui indique le **tempo** d'une chanson, pas les battements cardiaques.

BPM en musique - Explication

Le BPM mesure la **vitesse** d'une chanson :

- **BPM élevé** = musique rapide, énergique
- **BPM faible** = musique lente, calme

Exemples concrets

- **Sport (120-180 BPM)** : Musique électro, rock énergique pour garder le rythme
- **Détente (60-100 BPM)** : Ballades, jazz doux, musique d'ambiance
- **Fête (110-140 BPM)** : Pop dansante, house, musique de club
- **Étude (70-110 BPM)** : Lo-fi, classique, ambient - assez lent pour ne pas distraire